Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

THE EVOLUTION OF MACHINE LEARNING AND AI

Week 7 Core Booklet

Large Language Models, and Your First Project Step

Module	The Evolution of Machine Learning and AI (5 ECTS, Semester 1)
Week	Week 7 of 12
Reading time	About 45 minutes for Chapters 1 to 3. Chapters 4 to 7 are step-by-step guides: keep them open while you do the lab at home.
Goes with	The Week 7 lecture · the notebook `capstone_step1.ipynb` · the Week 7 form on Tutors

What this booklet is. The required reading for Week 7, in the same order as the lecture. Chapters 1 to 3 explain the topic and your project step. Chapters 4 to 7 assume you have **never used** Google Colab, Python or Ollama, and walk you through every click.

Contents

1 Large language models . . . 3
2 How this connects to your project . . . 8
3 This week's project step . . . 10
4 Getting started with Google Colab . . . 13
5 The Python you need . . . 15
6 Using this week's notebook . . . 16
7 Ollama and tokens (Part A at home) . . . 18
8 Check yourself . . . 19

YOUR WEEK, IN ORDER

1	The lecture (or its recording).	2 h
2	This booklet, Chapters 1 to 3.	45 min
3	Lab at home, Part A: install Ollama, count tokens, compare two models (Chapter 7).	15 min
4	Lab at home, Part B: your law and its answer sheet in Google Colab (Chapters 4 to 6).	45 min
5	Hand in the Week 7 form on Tutors.	5 min

Stuck on anything for 40 minutes? Stop, and post in the channel.

1 Large language models

COVERS	Lecture Part 1
TIME	20 minutes
IN BRIEF	A large language model guesses the next word, one word at a time, extremely well. It is built in three stages. Bigger models have kept getting better, but the cost of each step up is now enormous.

WORDS IN THIS CHAPTER, IN PLAIN ENGLISH

LLM	large language model: a very large machine that works with text
token	a piece of text the model reads: a word, part of a word, or punctuation
transformer	the design inside every LLM; its attention step lets every word look at every other word
pretraining	reading a huge amount of text and practising guessing the next word
fine-tuning	practising one particular job afterwards
RLHF	learning from human feedback: people rate answers, the model learns from the ratings
open model	one you can download and run on your own computer
scaling	making models bigger, with more data and more computing

1.1 What a large language model does

A large language model (LLM) does one thing: it reads some text and guesses the **next word**. It gives every possible next word a score for how likely it is, picks one (usually a likely one), adds it to the text, and does it all again. A long answer is just this, repeated hundreds of times, very fast.

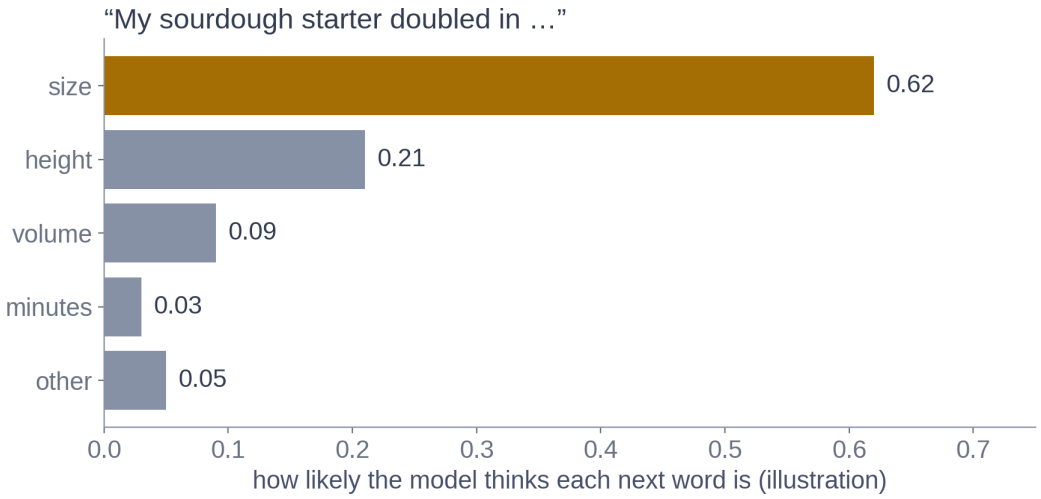


Figure 1: After “My sourdough starter doubled in”, the model rates possible next words. The numbers are an illustration, not from a real model.

There is no separate part for answering questions, writing code or summarising. It is all the same skill, done extremely well. Because the model has read so much text, good guesses about the next word end up looking like understanding.

1.2 What is inside: a transformer

Inside every LLM is a **transformer**, which you met in Week 4. Its key step is **attention**: every word looks at every other word and decides which ones matter. In “The starter rose because it was warm”, attention lets the word “it” work out that it means the starter.

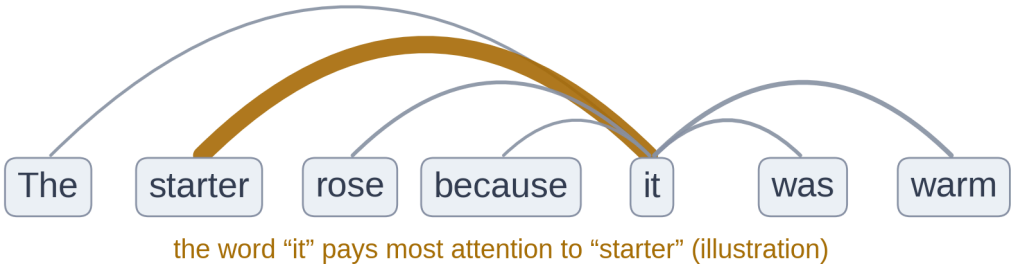


Figure 2: Attention: the word “it” looks at every other word; the thickest line goes to “starter”. An illustration.

Remember this picture: *every word looks at every other word*. Next week you will see that this is also why language models become slow and expensive when the text is very long.

1.3 How an LLM is made: three stages

EVERYDAY ANALOGY

Think of an apprentice chef. First they read every cookbook in the library. Then they train in one kitchen, learning one job. Then they learn from what the customers send back.

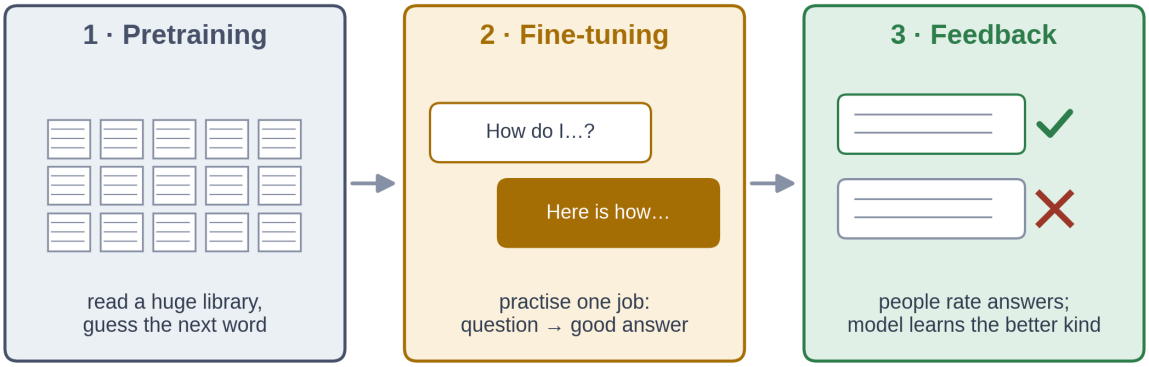


Figure 3: The three stages: read a huge library; practise one job; learn from people’s ratings.

1. **Pretraining.** The model reads a huge amount of text (websites, books, code) and practises guessing the next word, billions of times, adjusting its internal numbers each time it is wrong. This is Week 3′s guess, check, blame, nudge loop at an enormous scale. It runs for weeks on thousands of computer chips, and it is where almost all of the cost goes. The model picks up grammar, facts and styles of writing as a side effect.

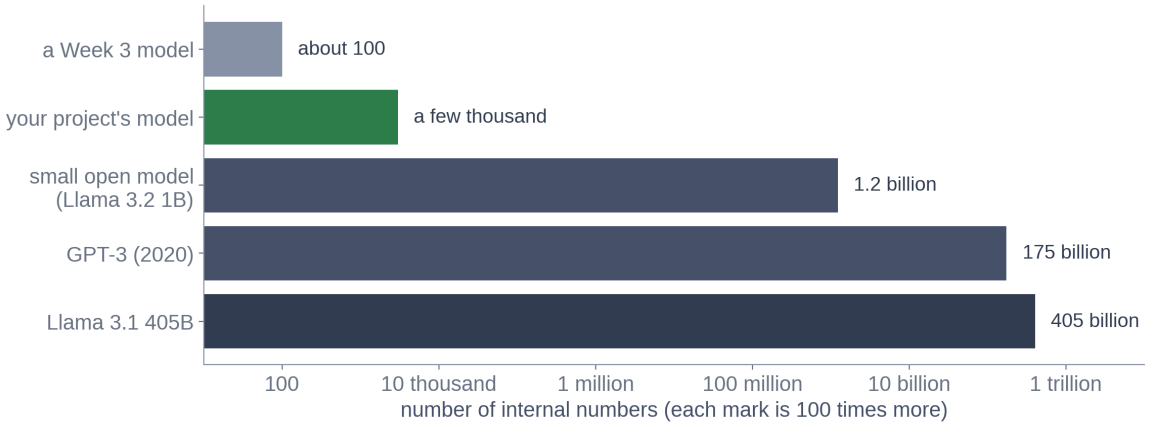


Figure 4: How many internal numbers different models have. Each mark along the bottom is 100 times more. Your project's model is in green.

1. **Fine-tuning.** The model practises one particular job, usually being a helpful assistant, on many examples of good questions with good answers.
2. **Learning from feedback (RLHF).** People compare pairs of answers and say which is better; the model is nudged towards the kind people prefer.

1.4 Tokens

Before a model reads anything, the text is cut into **tokens**: whole words, parts of words, spaces and punctuation. Common words are usually one token; longer or rarer words are cut into pieces. In English, one token is about three-quarters of a word on average, so 1,000 tokens is roughly 750 words.

Tokens matter for three reasons:

- **Cost and speed.** Companies charge per token, and longer text takes longer.
- **A length limit.** Each model can only read a certain number of tokens at once.
- **Odd mistakes.** The model sees chunks of words, not letters. That is why language models have often miscounted the letter r in "strawberry".

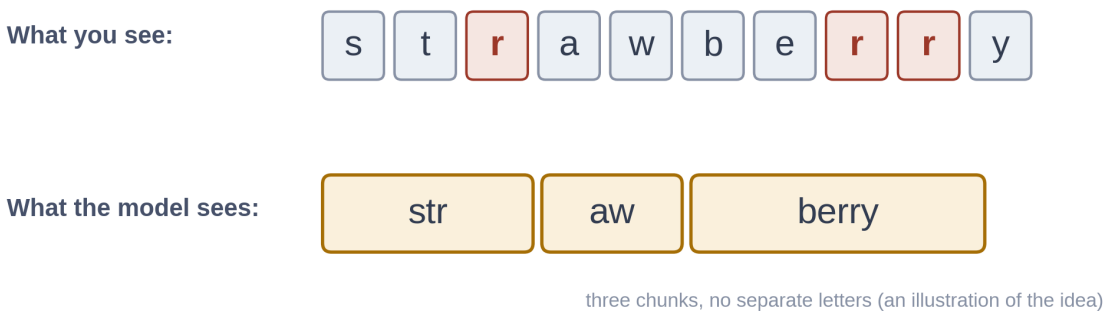
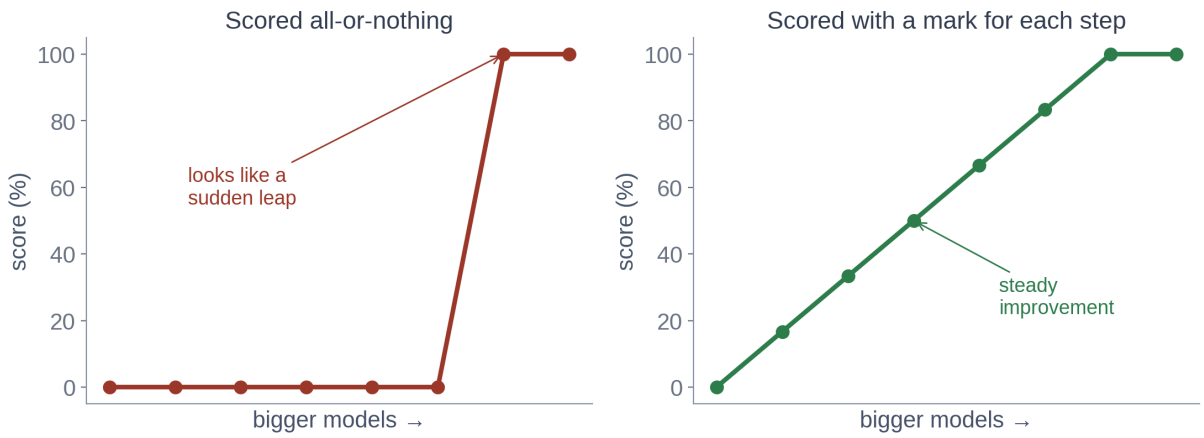


Figure 5: You see ten letters; the model sees a few chunks. An illustration of the idea.

1.5 Emergent abilities, and the argument about them

Some skills, such as arithmetic with several steps, seemed to appear suddenly once models were big enough, so people called them **emergent abilities**. Other researchers pointed out that many tests are scored all-or-nothing; measured with partial credit, the improvement often looks gradual.



The same models, on the same three-step sums. Only the scoring differs. (Illustration.)

Figure 6: The same models on the same three-step sums. Scored all-or-nothing, the score leaps; with a mark for each step, it climbs steadily. An illustration.

The argument is not settled. The habit to keep is a Week 1 habit: when you read that a model "suddenly" gained a skill, ask how the skill was measured.

1.6 Open models and closed models

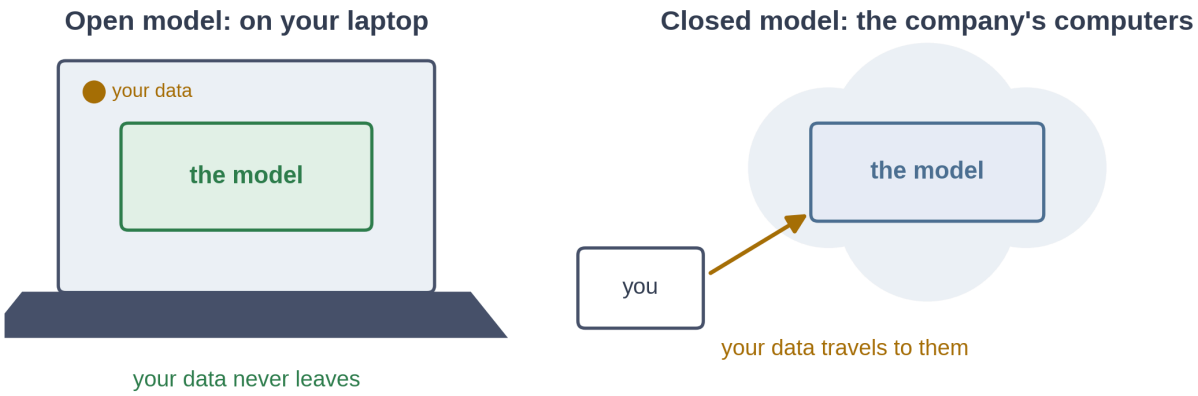


Figure 7: An open model runs on your laptop and your data stays there; a closed model runs on the company's computers and your data travels to them.

	Open (Llama, DeepSeek)	Closed (ChatGPT)
Where it runs	on your own computer	on the company's computers
Your data	stays with you	goes to the company
Cost	free to download	free tier, or pay
Ability	small ones are weaker	usually stronger
Control	you can change it	you use it as it is

Neither is simply better. Choose by what the job needs: a hospital might choose an open model to keep patient data private; a student writing a cover letter might happily use a closed one. In Part A of the lab at home you try both.

1.7 Scaling, and where it strains

For about ten years, the recipe was simple: more data, more computing power and a bigger model gave a better model. The improvement followed a regular pattern that researchers called **scaling laws**. But each step up now gives a smaller gain for a much larger cost.

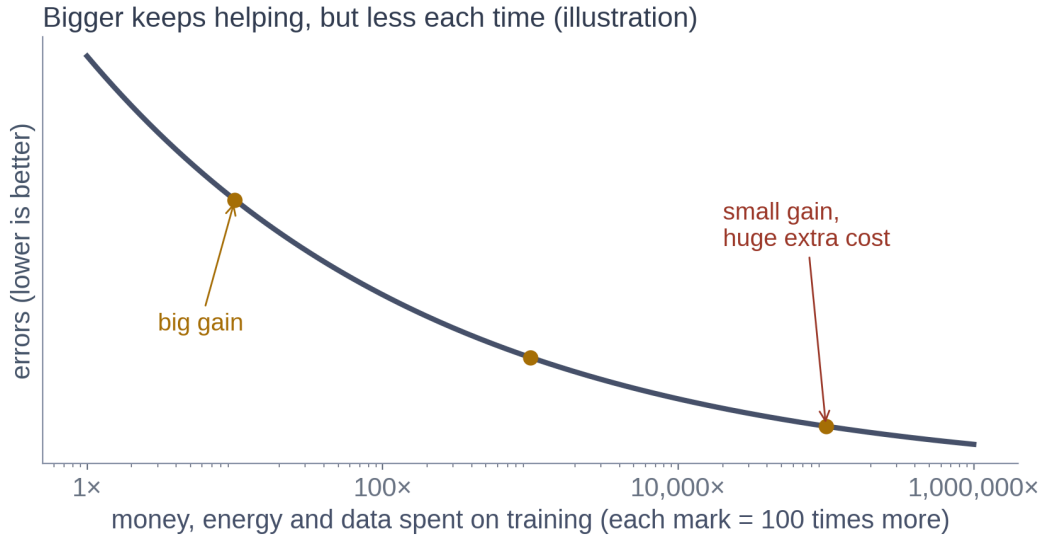


Figure 8: The shape of diminishing returns: each step to the right costs a hundred times more, and gains less. An illustration, not real data.

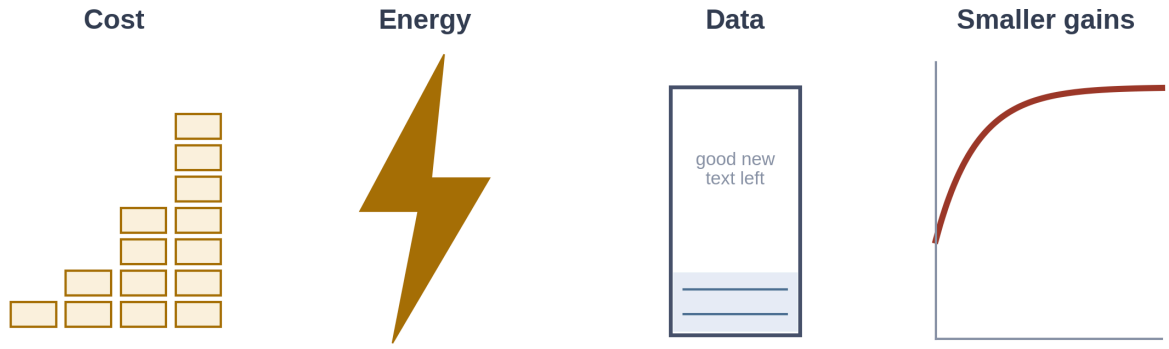


Figure 9: The four walls: cost, energy, data, smaller gains.

Wall	What it means
Cost	Training one top model is reported to cost over $100 million.
Energy	Training and running these models uses electricity on the scale of power stations.
Data	Good-quality new text on the internet is running out.
Smaller gains	Each step up improves the model less than the one before.

Weeks 8 and 9 look at the two answers the field is turning to: **faster** models and **smaller** models.

★ KEY POINTS

- An LLM guesses the next token, one at a time, using a transformer.
- It is made in three stages: pretraining, fine-tuning, learning from feedback.
- Tokens explain cost, length limits and some odd mistakes.
- Open models run on your computer; closed models run online. Choose by the job.
- Bigger has worked, but cost, energy, data and smaller gains are now walls.

2 How this connects to your project

COVERS Lecture Part 2
TIME 10 minutes
IN BRIEF Language models learn from billions of examples. Your project learns from one law of science, with no data. That is exactly why it is interesting now.

2.1 Two opposite ways to learn

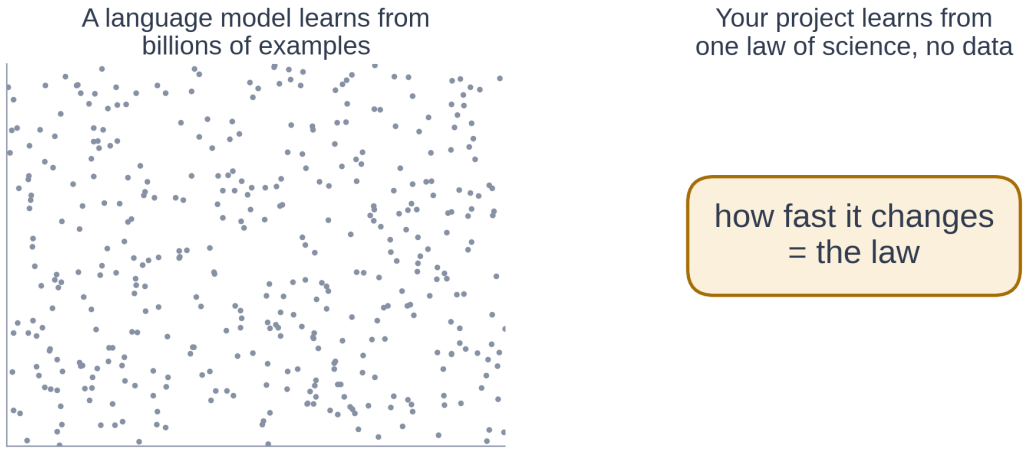


Figure 10: Two ways for a machine to learn: from billions of examples, or from one law.

A language model knows only what it has picked up from its data. Your project learns from a law of science instead, by checking its guesses against the law, as in Week 5. When data is scarce or expensive (a new machine, a rare illness, an experiment that costs a fortune), a law can do what data cannot.

2.2 Why this matters right now

The four walls are all about size. A language model has billions of internal numbers; your model will have a few thousand. It trains in a minute or two and needs no data. The walls the big companies are hitting are exactly the walls your kind of model avoids, which is one reason researchers are interested in models that know the laws of science. In Week 9 you will measure exactly how small and fast yours is.

2.3 Use a language model as a helper, then check it

Good uses for your project: ask a language model to explain your law in plain words, to explain an error message, or to suggest where a number might come from. Then check what it says against the student guide, a textbook or a reliable website.

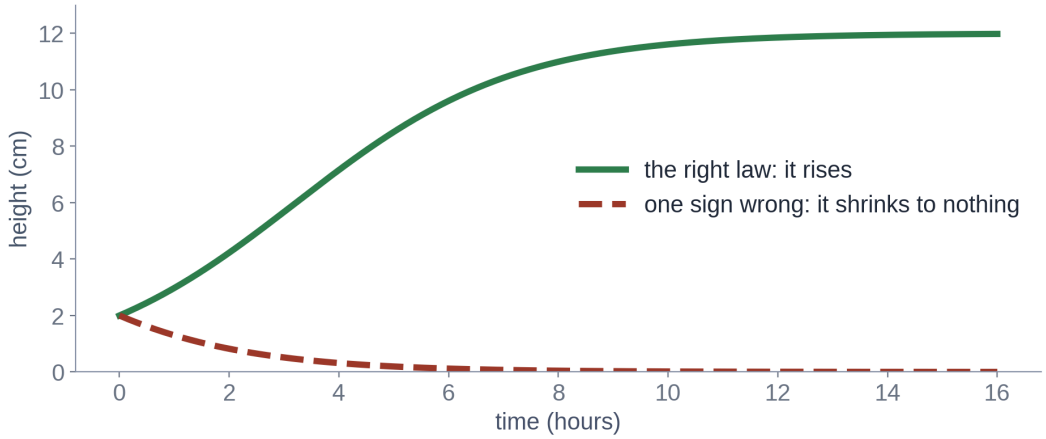


Figure 11: One wrong sign, the kind of slip a language model can make, and the sourdough starter shrinks to nothing instead of rising.

EVERYDAY ANALOGY

A language model is like a confident friend who has read a lot: often right, sometimes wrong, and just as sure either way. Check, do not trust.

3 This week's project step

COVERS Lecture Parts 2 and 3
TIME 15 minutes
IN BRIEF Write down four things so a computer can use them: your law as one line of code, its numbers, its start, and its answer sheet. The class example is a sourdough starter, which is not one of the project ideas.

3.1 The four things

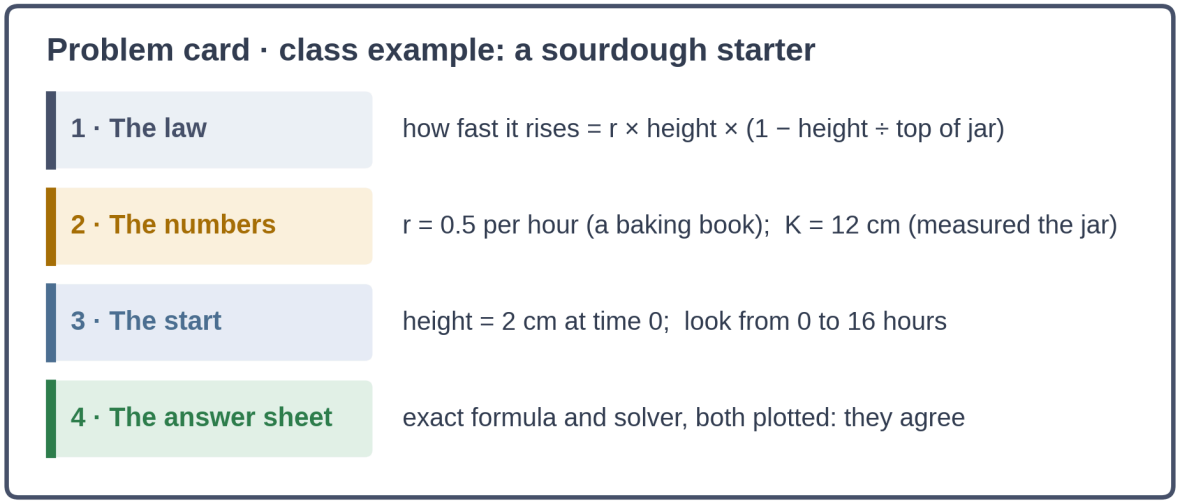


Figure 12: A filled-in problem card for the class example. You make one like it for your own law.

What	What it means
1 · The law	One line of code that says how fast your thing changes.
2 · The numbers	Each number in the law, with its unit and where you found it.
3 · The start	Where your thing begins; for laws along a line, also what happens at the two ends.
4 · The answer sheet	The right answer, worked out another way, to test your model against from next week.

3.2 The class example: a sourdough starter

A sourdough starter is flour and water with live yeast in it. Once fed, it rises fast at first, then slows down as it gets near the top of the jar.

✎ WORKED EXAMPLE: the law, in words and as code

In words: *how fast it rises = r × height × (1 − height ÷ top of the jar).*

When the starter is small, the bracket is close to 1, so the bigger it is, the faster it grows. Near the top of the jar, the bracket is close to 0, so it slows down and stops. As code, the same sentence:

```
def rate(t, H):                  # how fast the height changes
    return r * H * (1 - H / K)   # the law, as one line of code
```

What	Value	Unit	Where it came from
growth rate r	0.5	per hour	a baking book
top of the jar K	12	cm	measured the jar
start H0	**2**	**cm**	**measured after feeding**



Figure 13: A typo in one number (5 instead of 0.5) still runs, but answers the wrong question. Nothing warns you.

IN PLAIN ENGLISH

Choose units that keep your numbers close to 1: hours rather than seconds, centimetres rather than metres. A growth rate of 0.5 per hour is far friendlier than 0.00014 per second, and models learn much better that way.

3.3 Which shape is your law?

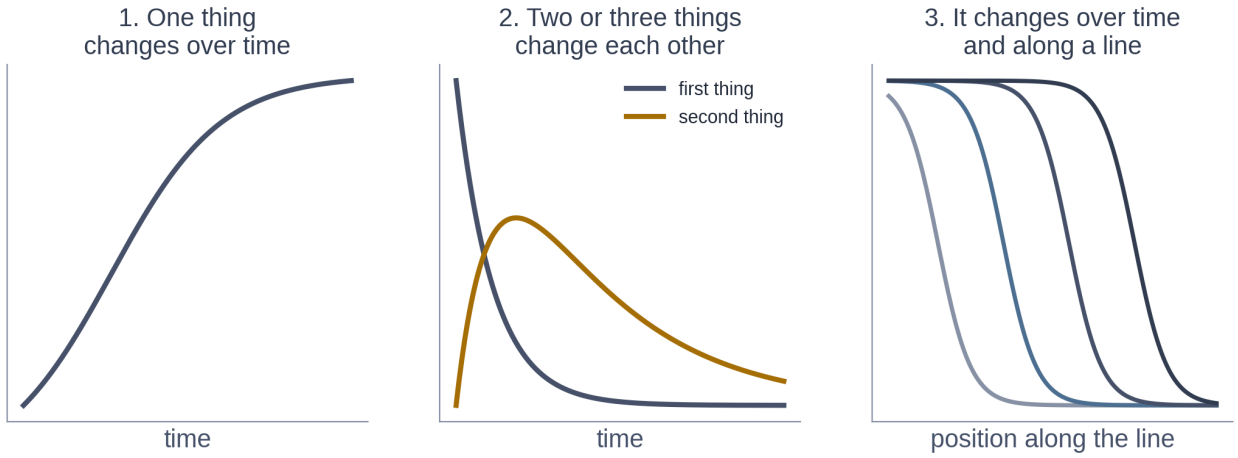


Figure 14: The three shapes a law can take.

Idea form, question 18	Shape	Notebook part
small	1 · one thing changes over time	Part 2
medium	2 · two or three things change each other	Part 3
large	3 · it changes over time and along a line	Part 4

If your law talks about how fast the *speed* changes, treat it as two things, the position and the speed: that is shape 2.

3.4 The answer sheet

The answer sheet is the right answer, worked out another way. Some laws have an **exact formula** (your student guide says which); otherwise, a **solver** works the answer out in many small steps. If you can, make both: they should lie on top of each other.

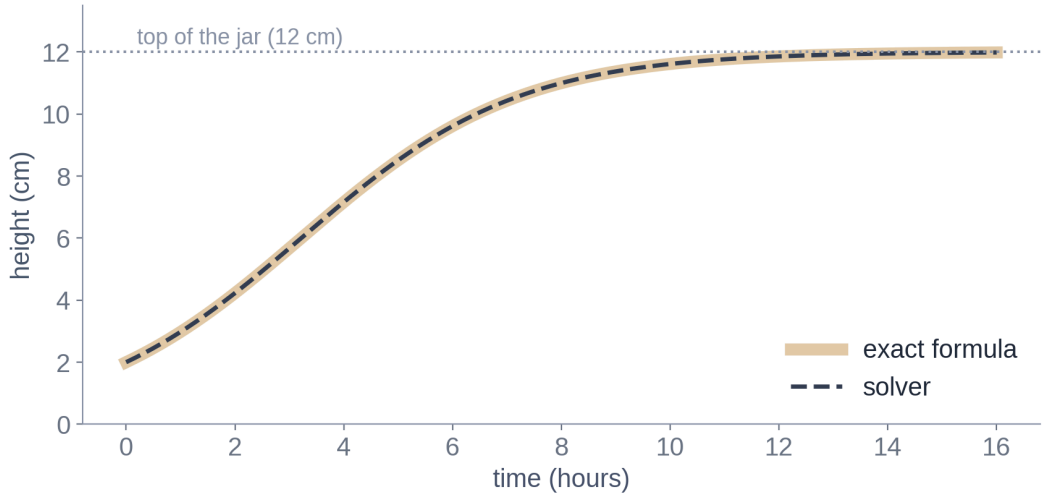


Figure 15: The sourdough starter's answer sheet: the exact formula (thick line) and the solver (dashed line) agree.

> **TRY IT YOURSELF**
>
> On paper: (1) say what changes in your project, in a few words; (2) write your law as "how fast it changes = …"; (3) show it to someone. Can they follow it? The "In plain words" column of the student guide is a good start.

4 Getting started with Google Colab

COVERS	Lab at home, Part B
TIME	10 minutes, once
IN BRIEF	Google Colab runs Python in your web browser, free, with nothing to install. You only need a Google account.

4.1 What Colab is

Google Colab is a free website where you can run Python code in your browser. It works with **notebooks**: pages that mix ordinary text with small boxes of code called **cells**. To run a cell, click inside it and press **Shift + Enter**, or click the round ▸ button at its left. The result appears just underneath.

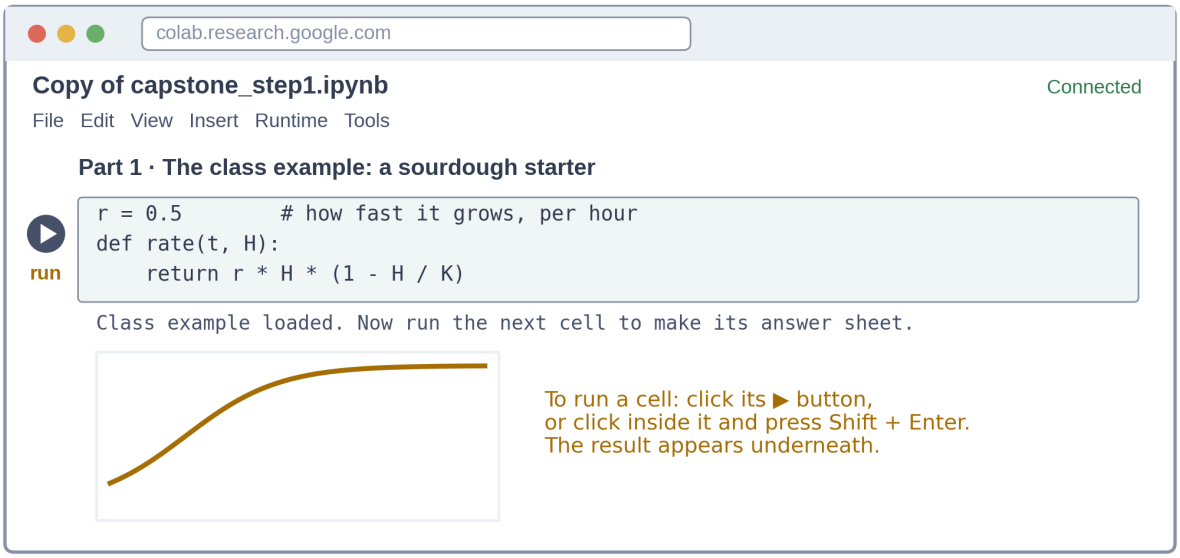


Figure 16: Roughly what a Colab notebook looks like: a heading, a code cell with its run button, and its result underneath.

4.2 Step by step: open this week's notebook

1. On Tutors, open the Week 7 lab and download the Week 7 lab file. It is a zip file: unzip it (Windows: right-click, **Extract All** Mac: double-click). You now have `capstone_step1.ipynb`.
2. Open Chrome (or another browser) and go to **colab.research.google.com**.
3. Sign in with a Google account (top right). A personal Gmail account is fine.
4. A window called *Open notebook* appears. Click **Upload**, then **Browse**, and choose `capstone_step1.ipynb`. If no window appears, click **File**, then **Upload notebook**.
5. The notebook opens. Click **File**, then **Save a copy in Drive**. A new tab opens called *Copy of capstone_step1.ipynb*. Work in this copy from now on; Colab saves it in your Google Drive.
6. Click inside the first code cell (Step 0) and press **Shift + Enter**.
7. If a warning says *This notebook was not authored by Google*, click **Run anyway**. It is the module's notebook from Tutors.
8. Top right, it says *Connecting* for a few seconds. Then the message *Ready. Now run the next cell.* appears under the cell.

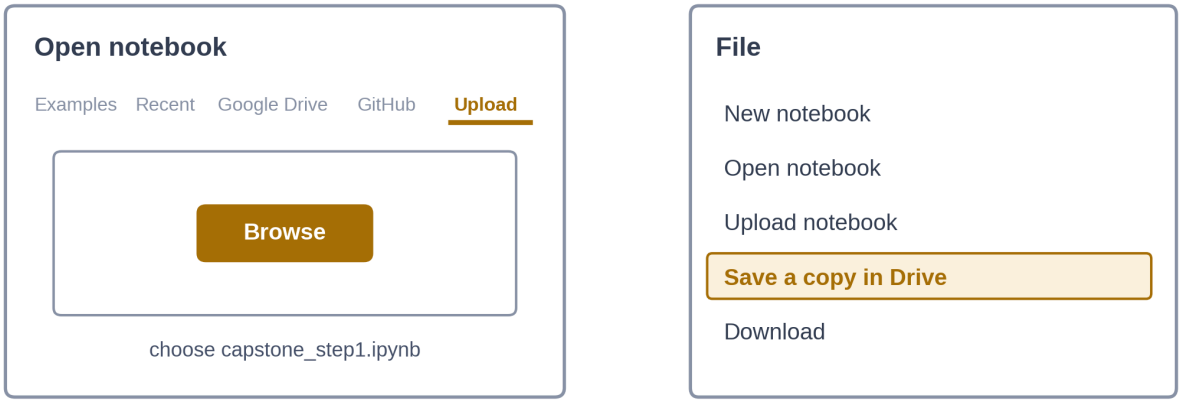


Figure 17: Left: the Open notebook window, with Upload and Browse. Right: the File menu, with Save a copy in Drive.

4.3 Coming back later

Go to **colab.research.google.com**, or to **drive.google.com** and open the folder *Colab Notebooks* your copy is there. Each time you come back, run the cells again from the top, because Colab forgets everything when you close it.

> **COMMON MISTAKE**
>
> Working in the original instead of the copy, then losing your changes. Always use **File → Save a copy in Drive** first.

5 The Python you need

COVERS Lab at home, Part B
TIME 10 minutes
IN BRIEF You do not write programs in this module. You read a few lines and change the ones marked `<--`. Five things are enough.

5.1 Five things

You will see	It means
`# a note`	A note for humans. The computer ignores everything after `#` on that line.
`r = 0.5`	Store the number 0.5 under the name `r`. Later, `r` means 0.5.
`def rate(t, H):`	The start of a rule called `rate`, which needs a time `t` and a height `H`.
`    return ...`	What the rule gives back. The line is pushed in (indented); leave indents as they are.
`* / ** np.sqrt np.exp`	Times, divide, to the power of, square root, *e* to the power of.

5.2 Writing your law as code

Say your law in words first, then swap the words for names. For example, "how fast it changes = minus a times the square root of the value" becomes `return -a * np.sqrt(y)`. Brackets work as in maths: `r * H * (1 - H / K)` means r times H times (1 minus H divided by K).

5.3 When something goes wrong

Red text under a cell is an **error message**. Read its **last line**.

Last line says	Usually means
`SyntaxError`	A typo: a missing bracket, a missing colon after `def ...`, or a stray character.
`NameError: name 'x' is not defined`	You used a name that does not exist yet: a spelling mistake, or you skipped running an earlier cell.
`IndentationError`	A line is pushed in too much or too little. Match the lines around it.

If you are stuck: click **Runtime → Restart session**, run the cells again from the top, and if it still fails, ask a language model what the error means (then check), or post in the channel.

6 Using this week's notebook

COVERS Lab at home, Part B
TIME 25 minutes
IN BRIEF Run the class example, then put your own law in the part that matches its shape, and download your answer-sheet picture.

Part	What it does	Who runs it
Step 0	Loads the tools	everyone
Part 1	The class example: a sourdough starter	everyone (just run it)
Part 2	Your law, if one thing changes over time	most of you
Part 3	Your law, if two or three things change each other (template: two connected water tanks)	only if it is your shape
Part 4	Your law, if it changes over time and along a line (template: a species spreading along a coast)	only if it is your shape

6.1 Step by step

1. Run Step 0 and the two cells of Part 1. You should see the sourdough graph: two lines on top of each other, rising from 2 cm and levelling off at 12 cm. Underneath: a difference of 0.0 cm, and a leftover of about 0.01.
2. Go to the part that matches your law's shape. Each line you change ends with `<--`.
3. The cell starts with a placeholder law, so it runs straight away. Replace each line marked `<--`: the name of what changes and its unit, your unit of time, your numbers (with unit and source in the note), your start, how long to look, and your law line.
4. If your law has an exact formula, set `HAS_EXACT = True` and write the formula; if not, set `HAS_EXACT = False`.
5. Press **Shift + Enter** in your cell, then in the cell below it. Your answer sheet appears.
6. If you made both, check the printed difference between formula and solver is close to zero. If not, check your law line and your formula.
7. Click the **folder icon** on the left, find `my_answer_sheet.png`, click its **three dots**, then **Download**.

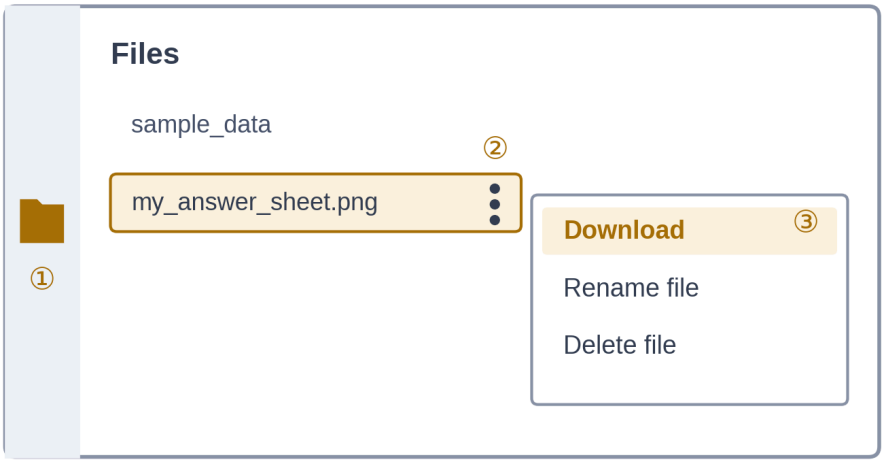


Figure 18: Downloading your picture: the folder icon (1), the three dots beside the file (2), then Download (3).

COMMON MISTAKE

Running cells out of order. If you change something near the top, run every cell below it again.

7 Ollama and tokens (Part A at home)

COVERS	Lab at home, Part A
TIME	15 minutes, mostly waiting for a download
IN BRIEF	Install a free program that runs a small open language model on your own laptop, count the tokens in a sentence, and compare two models' answers about your law.

7.1 Install Ollama and run a small model

Windows	Mac
1. Go to **ollama.com**, click **Download**, choose **Windows**, and run the downloaded file. Ollama starts in the background. 2. Press the Start key, type `cmd`, press Enter. A black window opens: the Command Prompt. 3. Type `ollama run llama3.2:1b` and press Enter. The first time, it downloads about 1.3 GB. 4. When you see `>>>`, type your question and press Enter. 5. To leave, type `/bye` and press Enter.	1. Go to **ollama.com**, click **Download**, choose **macOS**, open the downloaded file, move Ollama to Applications, and open it. 2. Press Cmd + Space, type `Terminal`, press Enter. 3. Type `ollama run llama3.2:1b` and press Enter. The first time, it downloads about 1.3 GB. 4. When you see `>>>`, type your question and press Enter. 5. To leave, type `/bye` and press Enter.

IN PLAIN ENGLISH

Old or small laptop? Use `ollama run gemma3:1b` instead: about 815 MB. Do the download on a good connection, not on mobile data. Next time, the same command starts the model in a few seconds, with no internet needed.

7.2 Count tokens

Go to **platform.openai.com/tokenizer**. Type one sentence about your project. The page shows each token in a different colour and counts them. Compare the number of tokens with the number of words.

7.3 Compare two models

Ask the same question to the model in Ollama and to ChatGPT (or Copilot): *"Explain [your law] in two sentences."* Then check both answers against your student guide. Note one difference between them, and anything either one got wrong.

8 Check yourself

COVERS Revision
TIME 10 minutes
IN BRIEF Cover the right-hand column, answer, uncover.

#	Question	Answer
1	In one sentence, what does a large language model do?	It guesses the next word (strictly, the next token), one at a time, extremely well.
2	What does attention let a transformer do?	Let every word look at every other word and decide which ones matter.
3	Name the three stages of making an LLM.	Pretraining, fine-tuning, learning from human feedback (RLHF).
4	Which stage costs the most?	Pretraining.
5	Roughly how many words are 1,000 tokens?	About 750.
6	Why might a model miscount the letters in a word?	It sees tokens (chunks of words), not individual letters.
7	Give one reason to choose an open model.	Your data stays on your computer; it is free; you control it.
8	Name the four walls scaling is hitting.	Cost, energy, data, smaller gains.
9	How does your project learn differently from an LLM?	From one law of science, with no data, instead of billions of examples.
10	What four things do you write down this week?	The law as one line of code, its numbers, its start, its answer sheet.
11	What is the answer sheet for?	To test your model against the right answer, worked out another way.
12	Your formula and solver lines do not agree. What do you check first?	Your law line and your formula, for a typo or a wrong sign.